

that passing `['ls -al']` wouldn't work. We store the command's output as a string in `outputStr` ❷. Online documentation for `subprocess.run()` and `locale.getdefaultlocale()` will give you a better idea of how these functions work, but they make the code work on any operating system running Python.

## ***Minimizing Typing with Tab Completion***

Because advanced users enter commands into computers for hours a day, modern command lines offer features to minimize the amount of typing necessary. The *tab completion* feature (also called command line completion or autocomplete) lets a user type the first few characters of a folder or filename and then press the TAB key to have the shell fill in the rest of the name.

For example, when you type `cd c:\u` and press TAB on Windows, the current command checks which folders or files in `C:\` begin with `u` and tab completes to `c:\Users`. It corrects the lowercase `u` to `U` as well. (On macOS and Linux, tab completion doesn't correct the casing.) If multiple folders or filenames begin with `U` in the `C:\` folder, you can continue to press TAB to cycle through all of them. To narrow down the number of matches, you could also type `cd c:\us`, which filters the possibilities to folders and filenames that begin with `us`.

Pressing the TAB key multiple times works on macOS and Linux as well. In the following example, the user typed `cd D`, followed by TAB twice:

---

```
al@al-VirtualBox:~$ cd D
Desktop/  Documents/ Downloads/
al@al-VirtualBox:~$ cd D
```

---

Pressing TAB twice after typing the `D` causes the shell to display all the possible matches. The shell gives you a new prompt with the command as you've typed it so far. At this point, you could type, say, `e` and then press TAB to have the shell complete the `cd Desktop/` command.

Tab completion is so useful that many GUI IDEs and text editors include this feature as well. Unlike command lines, these GUI programs usually